By: Georges Sfeir

Advanced Test-Benching in SystemVerilog

Verifying Hardware Behavior in Simulation

SystemVerilog Assertions, Coverage, & Constrained Random Verification

What Are SystemVerilog Assertions?

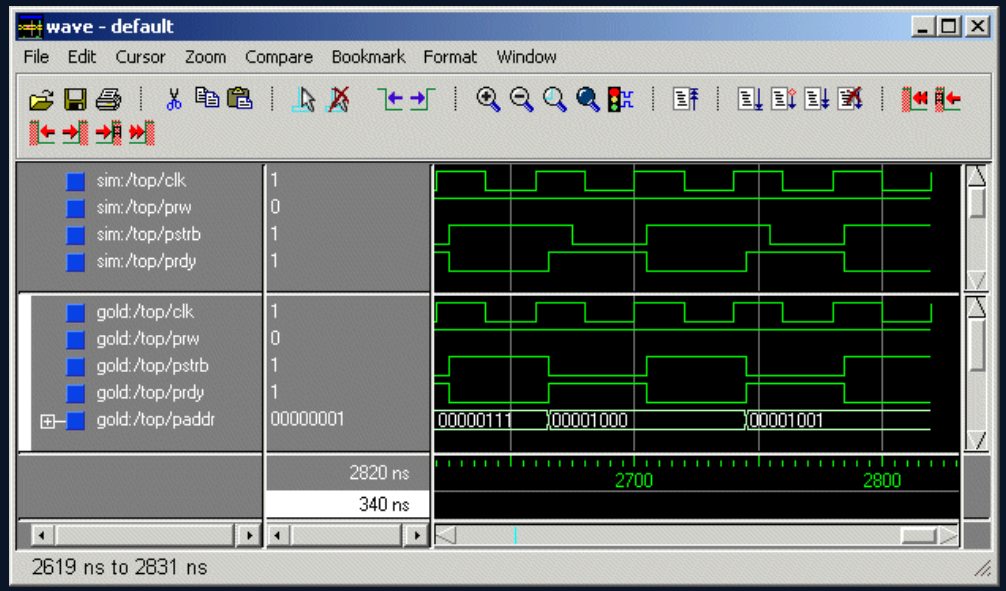
"Instead of manually checking outputs, assertions do the watching for you."

IMMEDIATE

Checked like an if statement at a point in time

CONCURRENT

Checked across time, triggered by a clock edge



Your First Concurrent Assertion

Checking Behavior Across Clock Cycles

Core idea: if condition A is true now, condition B must be true one cycle later

```
// If reset is high, output must be 0 on the next cycle
assert property (@(posedge clk) rst |=> out == '0);

// If input is driven, output must match it one cycle later
assert property (@(posedge clk) disable iff (rst) in |=> out);
```

`|=>`

means “on the next clock cycle”

`disable iff`

suspends the check during reset

The $past and $stable Functions

Looking Back in Time

Sometimes you need to compare the current output to what the input used to be

```
// Output should equal whatever 'in' was one cycle ago
assert property (@(posedge clk) disable iff (rst)
    en |=> out == $past(in, 1));

// Output shouldn't change when enable is low
assert property (@(posedge clk) disable iff (rst)
    !en |=> $stable(out));
```

$past(signal, N)

Returns the value of signal N cycles ago

$stable(signal)

Returns true if signal didn't change from the previous cycle

COMPLETE TESTBENCH

A Complete Register Testbench in 3 Lines

Assertions Make Testbenches Compact & Powerful

```
// Capture: when enabled, output follows input next cycle
assert property (@(posedge clk) !rst && en |=> out == $past(in, 1));

// Hold: when not enabled, output should not change
assert property (@(posedge clk) disable iff (rst) !en |=> $stable(out));

// Reset: output must go to 0 after reset
assert property (@(posedge clk) rst |=> out == '0);
```

01 CAPTURE

en high and rst low → output follows input on the next cycle

02 HOLD

en low → output must not change

03 RESET

rst high → output goes to 0

⚠️ PITFALLS

Watch Out — Subtle Pitfalls!

Common Assertion Gotchas with $past During Reset

Problem: $past tracks inputs during reset — the cycle right after reset deasserts can cause false failures

```
// ❌ Can fail on the first cycle after reset
assert property (@(posedge clk) disable iff (rst) out == $past(in, 1));

// ✅ Use implication to skip that first tricky cycle
assert property (@(posedge clk) disable iff (rst) 1 |=> out == $past(in, 1));

// ✅ Or use !rst as the trigger instead of disable iff
assert property (@(posedge clk) !rst |=> out == $past(in, 1));
```

Key rule: If you clear reset on a clock edge, use $sampled(rst) in your disable condition or wait an extra cycle.

What Is Coverage & Why Does It Matter?

"Assertions tell you when something goes wrong — coverage tells you if you tested enough."

- Assertion properties flag failures
- Cover properties track whether specific scenarios were ever tested
- Without proper coverage, your testbench might never hit the edge cases that break real hardware

```
// Did we ever test a write? A read?
cp_write : cover property (@(posedge clk) wr_en);
cp_read  : cover property (@(posedge clk) rd_en);
```

ASSERTIONS

Flags failures

Signals a violation automatically

COVERAGE

Tracks completeness

Reports how much of the space was exercised (new Questa window #epic)

Cover Properties — Testing the Edge Cases

Make sure your testbench actually exercises dangerous scenarios

```
// Did we ever try to write when the FIFO was already full?
cp_write_while_full :
    cover property (@(posedge clk) wr_en && full);

// Did we ever try to read when the FIFO was empty?
cp_read_while_empty :
    cover property (@(posedge clk) rd_en && empty);
```

- If these never trigger, your testbench has a blind spot
- Cover properties report a count — you want that count to be > 0

Count = 0 → Never tested

Count > 0 → Scenario exercised

Covergroups & Coverpoints

Cover properties check specific events. Covergroups track the range of values tested.

```
covergroup cg @(posedge clk);
    // Tracks whether carry_in was ever 0 and ever 1
    cin  : coverpoint carry_in;
    cout : coverpoint carry_out;
    // Divides all possible values of in0/in1 into 64 bins
    // and tracks how many were hit
    in0_bin : coverpoint in0;
    in1_bin : coverpoint in1;
endgroup
```

- By default, up to 64 bins are auto-created per coverpoint
- Each bin represents a range of values — were they all hit?

Bins hit:

✅ hit ❌ missed

CROSS COVERAGE

Cross Coverage & Custom Bins

Check combinations of inputs, not just individual values

```
covergroup cg @(posedge clk);
    // Ensure in0 is tested at 0 and max, at least 10 times each
    in0_extremes: coverpoint in0 {
        bins zero = {0}; bins max_ = {'1}; option.at_least = 10;
    }
    // Divide into 16 bins, then cross them — 16x16 = 256 combos
    in0_full : coverpoint in0 { option.auto_bin_max = 16; }
    in1_full : coverpoint in1 { option.auto_bin_max = 16; }
    in0_cross_in1 : cross in0_full, in1_full;
endgroup
```

Cross coverage grid

- option.at_least sets a minimum hit count per bin
- Cross coverage catches bugs that only appear with specific input combinations

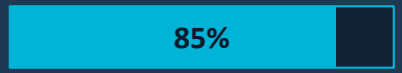
Checking Coverage Results

Print your coverage score at the end of simulation

```
cg cg_inst;
initial begin
    cg_inst = new(); // instantiate the covergroup
    // ... run your tests ...
    $display("Coverage = %0.2f %%", cg_inst.get_inst_coverage());
    disable generate_clock;
end
```

Tips for Questa

- Launch with *vsim –coverage* on ECE server to enable coverage tracking
- Click View → Coverage → Coverage Analysis to see your covergroups' hierarchy
- Aim for 100% coverage — if you can't reach it, add targeted tests



What Is Constrained Random Verification (CRV)?

"Instead of manually writing all test cases, let the simulator generate all the random inputs within rules you define."

```
class bit_diff_item #(WIDTH);
    rand bit [WIDTH-1:0] data;  // signal is randomized each time randomize() is called on object
    randc bit [WIDTH-1:0] cyclic_data; // random values are not repeated until every possible value has
endclass                               // been cycled through for cyclic_data

// Usage: generate a random test
bit_diff_item item = new;
if (!item.randomize()) $display("Randomize failed");
```

- CRV automatically generates diverse test inputs using the rand keyword and randomize()
- A transaction object represents an abstract input to the DUT — no timing, no signals, just data
- Constraints narrow the random space to meaningful values

P.S. you can call srandom([seed]) to override default random seed for RNG

Constraints — Guiding Randomness

Without constraints, random inputs may rarely hit important edge cases.

```
class bit_diff_item3 #(WIDTH);
    rand bit [WIDTH-1:0] data;
    rand bit go, en;
    // go should only be true ten percent of the time
    constraint c_go_dist {
        go dist {
            0 :/ 90,
            1 :/ 10
        };
    }
    // en should be true fifty percent of the time
    constraint c_en_dist {
        en dist {
            0 := 10,
            1 := 10
        };
    }
endclass
```

- dist sets a weighted probability across values
- Constraints let you model realistic or stress scenarios without hand-writing every case

Thanks for Coming!

Questions?